DMATH-CODE

Endliche Gruppen

Frühlingssemester 25

Serie 12

Lernziele

- Sie wissen was ein endliche Gruppe ist und können in zyklischen Gruppen Generatoren finden.
- Sie kennen den Unterschied zwischen symmetrische und asymmetrischer Verschlüsselung.
- Sie können die OTP- und RSA-Algorithmen durchführen und verstehen wie das Schnorr-Protokoll funktioniert.

Endliche Gruppen liefern eine mathematisch präzise Sprache zur Beschreibung von Symmetrien, Operationen und Strukturen, die in Algorithmen, Kryptographie, Codierungstheorie und der automatischen Verifikation zum Einsatz kommen. Besonders in der Kryptographie dienen Gruppen als Grundlage für viele moderne Verschlüsselungsverfahren wie RSA oder elliptische Kurven. Auch in der Theorie formaler Sprachen und bei der Modellierung von Automaten finden Gruppenstrukturen Anwendung. Die Untersuchung endlicher Gruppen ermöglicht es, wiederkehrende Muster zu erkennen und effizient zu nutzen – ein wesentliches Ziel der Informatik.

Frage 1: *Was sind endliche Gruppen?*

Wir wissen bereits, dass in der modularen Arithmetik schöne Eigenschaften, wie sie reelle oder rationale Zahlen haben, wieder vorhanden sein können. So besitzen zum Beispiel bestimmte ganze Zahlen in der modularen Arithmetik wieder multiplikativ inverse Zahlen:

Die Zahl 4 hat etwa in der modularen Arithmetik $\mathsf{mod}\,27$ die multiplikativ inverse Zahl 7, denn

$$4 \cdot 7 \,\mathsf{mod}\, 27 = 1 \ .$$

Allgemein sind für uns abstrakte Strukturen interessant, in denen alle Elemente ein Inverses haben:

Definition 1: Eine Menge G mit einer Operation $*$ heisst ***Gruppe***, wenn

- die Operation ***assoziativ*** ist:

$$a * (b * c) = (a * b) * c \quad \text{für alle Elemente } a, b, c \in G$$

- ein ***neutrales Element*** $n \in G$ existiert:

$$n * a = a * n = a \quad \text{für alle Elemente } a \in G$$

- jedes Element $a \in G$ ein ***inverses Element*** $a' \in G$ hat:

$$a * a' = a' * a = n \ .$$

Wenn die Menge G endlich viele Element hat, heisst die Gruppe ***endlich***.

Vertraute Gruppen sind zum Beispiel:

- Die reellen Zahlen $\mathbb{R}$ mit der Operation ***Plus***.
- Die reellen Zahlen ohne die Null $\mathbb{R}^*$ mit der Operation ***Mal***.
- Die rationalen Zahlen $\mathbb{Q}$ mit der Operation ***Plus***.
- Die rationalen Zahlen ohne $\mathbb{Q}^*$ die Null mit der Operation ***Mal***.
- Die ganzen Zahlen $\mathbb{Z}$ mit der Operation ***Plus***.

Diese Gruppen haben alle unendlich viele Elemente.

Für eine natürliche Zahl n bezeichnen wir mit

- $\mathbb{Z}_n$ die Menge aller Zahlen $0, 1, 2, 3, \ldots, n-1$
- $\mathbb{Z}_n^*$ die Menge aller Zahlen $a \in \mathbb{Z}_n$ mit $\mathsf{ggT}(a, n) = 1$.

In der modularen Arithmetik $\mathsf{mod}\, n$ haben wir endliche Gruppen:

- Die Zahlen $\mathbb{Z}_n$ mit der Operation ***Plus***.
- Die Zahlen $\mathbb{Z}_n^*$ mit der Operation ***Mal***.

Zum Beispiel gilt in der ***modularen Arithmetik*** $\mathsf{mod}\ 8$:

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

·	1	3	5	7
1	1	3	5	7
3	3	1	7	5
5	5	7	1	3
7	7	5	3	1

Es gibt überraschend viele endliche Gruppen und die Elemente müssen nicht Zahlen sein:

Aufgabe 1: Die Kinder hätten an der Wandtafel eine ***Operationstafel*** ergänzen sollen. Wie muss Lucky Luke die leeren Stellen füllen, damit eine endliche Gruppe entsteht?

In der Kryptographie sind endliche Gruppen von grosser Bedeutung, die eine zusätzliche Eigenschaft haben:

Definition 2: Eine Gruppe G heisst ***zyklisch***, wenn mindestens ein Element $g \in G$ alle anderen Element von G erzeugt:

$$G = \{n, g, g * g, g * g * g, \ldots, g^k, \ldots\} \ .$$

Solche Elemente g heissen ***Generatoren*** der Gruppe.

Zum Beispiel ist $\mathbb{Z}_8$ in der modularen Arithmetik **mod 8** zyklisch:

$$\begin{aligned}
\mathbb{Z}_8 : \ & 1 \xrightarrow{+1} 2 \xrightarrow{+1} 3 \xrightarrow{+1} 4 \xrightarrow{+1} 5 \xrightarrow{+1} 6 \xrightarrow{+1} 7 \xrightarrow{+1} 0 \ , \quad \textit{1 ist Generator} \\
& 2 \xrightarrow{+2} 4 \xrightarrow{+2} 6 \xrightarrow{+2} 0 \ , \quad \textit{2 ist kein Generator} \\
& 3 \xrightarrow{+3} 6 \xrightarrow{+3} 1 \xrightarrow{+3} 4 \xrightarrow{+3} 7 \xrightarrow{+3} 2 \xrightarrow{+3} 5 \xrightarrow{+3} 0 \ , \quad \textit{3 ist Generator} \\
& 4 \xrightarrow{+4} 0 \ , \quad \textit{4 ist kein Generator} \\
& 5 \xrightarrow{+5} 2 \xrightarrow{+5} 7 \xrightarrow{+5} 4 \xrightarrow{+5} 1 \xrightarrow{+5} 6 \xrightarrow{+5} 3 \xrightarrow{+5} 0 \ , \quad \textit{5 ist Generator} \\
& 6 \xrightarrow{+6} 4 \xrightarrow{+6} 2 \xrightarrow{+6} 0 \ , \quad \textit{6 ist kein Generator} \\
& 7 \xrightarrow{+7} 6 \xrightarrow{+7} 5 \xrightarrow{+7} 4 \xrightarrow{+7} 3 \xrightarrow{+7} 2 \xrightarrow{+7} 1 \xrightarrow{+7} 0 \ , \quad \textit{7 ist Generator}
\end{aligned}$$

Aber $\mathbb{Z}_8^*$ ist in der modularen Arithmetik **mod 8** nicht zyklisch:

$$\begin{aligned}
\mathbb{Z}_8^* : \ & 3 \xrightarrow{\cdot 3} 1 \ , \quad \textit{1 ist kein Generator} \\
& 5 \xrightarrow{\cdot 5} 1 \ , \quad \textit{5 ist kein Generator} \\
& 7 \xrightarrow{\cdot 7} 1 \ , \quad \textit{7 ist kein Generator}
\end{aligned}$$

Satz 1: In der modularen Arithmetik mit ***Primzahl p als Modulus*** ist

- $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$ eine ***zyklische*** Gruppe.
- Für jeden ***Generator g*** der Gruppe existieren also die Logarithmen

$$\log_g(a) \ \text{ für alle } a \in \mathbb{Z}_p^* \ .$$

Allgemein existieren für die Generatoren von zyklischen Gruppen alle Logarithmen!

$$\log_g(a) = x \ \text{ für } \ \underbrace{g * g * g * \ldots * g}_{x\text{-mal}} = a$$

Aufgabe 2:

a) Bestimmen Sie die Generatoren von $\mathbb{Z}_7^*$.

b) Bestimmen Sie mit einem Computer die Generator von $\mathbb{Z}_{17}^*$.

c) Ist die Gruppe auf der Wandtafel bei Lucky Luke zyklisch?

Frage 2: ***Was sind Kryptosysteme?***

Ein ***Kryptosystem*** ist ein ***öffentlich bekanntes Verfahren***, mit dem Klartexte in Chiffretexte transformiert werden und umgekehrt.

Das Verfahren ist also bekannt und kann von allen analysiert werden. Das Verdecken der Information im Klartext wird nur über die ***Geheimhaltung eines Schlüssels*** in der Anwendung des Systems erreicht (Kerckhoffs'sches Prinzip).

Wir unterscheiden symmetrische und asymmetrische Verfahren.

Definition 3: Ein ***Symmetrisches-Kryptosystem*** besteht aus einem

- probabilistischen Polynomzeit ***Schlüsselerzeugungsalgorithmus***

  $$k \leftarrow \texttt{KeyGen()}\ ,$$

  der einen gültigen Schlüssel k zurückgibt.
- probabilistischen Polynomzeit ***Verschlüsselungsalgorithmus***

  $$C \leftarrow \texttt{Enc(k,P)}\ ,$$

  der mit k den ***Klartext*** P verschlüsselt und einen ***Chiffretext*** C zurückgibt.
- deterministischen Polynomzeit ***Entschlüsselungsalgorithmus***

  $$P \leftarrow \texttt{Dec(k,C)}\ ,$$

  der mit k den ***Chiffretext*** C entschlüsselt und einen ***Klartext*** P zurückgibt.
- Das System heisst ***korrekt***, wenn für alle zulässigen Klartexte

  $$\texttt{Dec(k,Enc(k,P))} = \texttt{P} \quad \text{gilt.}$$

Das Vernam-One-Time-Pad - die perfekte Verschlüsselung

Das System transformiert ***Klartexte*** $P \in \mathbb{Z}_2^n$ wie folgt:

- `KeyGen()` erzeugt zufällig uniform einen ***Schlüssel*** $k \in \mathbb{Z}_2^n$
- `Enc(k,P)` $= P + k \bmod 2$ komponentenweise
- `Dec(k,C)` $= C + k \bmod 2$ komponentenweise

Aufgabe 3:

a) Verschlüsseln und Entschlüsseln Sie die Nachricht `1011010111` mit dem Schlüssel `1101100110`.

b) Zeigen Sie, dass das Vernam-One-Time-Pad korrekt ist.

c) Begründen Sie die perfekte Sicherheit des Vernam-One-Time-Pad. Es gilt demnach, dass Klartexte und Chiffretexte unabhängig sind.

Hinweis: Zeigen Sie $P(\texttt{Klartext}|\texttt{Chiffretext}) = P(\texttt{Klartext})$.

Das Vernam-One-Time-Pad ist in der Praxis unhandlich, weil der Schlüssel die gleiche Länge haben muss wie der Klartext. Damit der Empfänger den Chiffretext entschlüsseln kann, muss der den Schlüssel kennen (Schlüsselaustauschproblem). Dennoch:

- Es kann mit einem Pseudozufallsgenerator der Schlüssel k für das Vernam-One-Time-Pad generiert werden. Für so einen Pseudozufallsgenerator wird ein kürzerer Schlüssel gebraucht, aber die perfekte Sicherheit geht verloren.

- Auf einem grossen Speichermedium kann einmal jährlich per Kurier eine Kopie von Zufallsbits dem Kommunikationspartner zugestellt werden.

Der de-facto Standard weltweit für symmetrische Verschlüsselung ist ***AES*** (Advanced Encryption Standard) mit Schlüssellängen von 128, 192 oder 256 Bit. Dieses weit verbreitet Kryptosystem ist sehr effizient in Hardware und Software und wurde von Experten weltweit analysiert.

Definition 4: Ein ***Public-Key-Kryptosystem*** besteht aus einem

- probabilistischen Polynomzeit ***Schlüsselerzeugungsalgorithmus***

  $$(e, d) \leftarrow \texttt{KeyGen(k)}\ ,$$

  der ein gültiges Schlüsselpaar (e, d) in Abhängigkeit vom Sicherheitsparameter k zurückgibt.
  - e heisst ***öffentlicher Schlüssel***
  - d heisst ***privater Schlüssel***.
- probabilistischen Polynomzeit ***Verschlüsselungsalgorithmus***

  $$C \leftarrow \texttt{Enc(e,P)}\ ,$$

  der mit e den ***Klartext*** P verschlüsselt und einen ***Chiffretext*** C zurückgibt.
- deterministischen Polynomzeit ***Entschlüsselungsalgorithmus***

  $$P \leftarrow \texttt{Dec(d,C)}\ ,$$

  der mit d den ***Chiffretext*** C entschlüsselt und einen ***Klartext*** P zurückgibt.
- Das System heisst ***korrekt***, wenn für alle zulässigen Klartexte

  $$\texttt{Dec(d,Enc(e,P))} = \texttt{P} \quad \text{gilt.}$$

Bei einem Public-Key-Kryptosystem ist der öffentliche Schlüssel bekannt, er wird vom Sender zum Verschlüssen der Daten gebraucht.

Nur der Empfänger kann mit seinem geheimen privaten Schlüssel die verschlüsselte Nachricht wieder entschlüsseln.

Weil ein Klartext mehrfach mit dem gleichen öffentlichen Schlüssel verschlüsselt werden kann, muss der Algorithmus immer einen anderen Chiffretext zurückgeben. Sonst könnte ein Angreifer verschiedene Klartexte seiner Wahl verschlüsseln und mit einem Chiffretext vergleichen.

Der Verschlüsselungsalgorithmus muss probabilistisch sein!

Das RSA-Kryptosystem von Rivest, Shamir und Adleman

Das System transformiert ***Klartexte*** $P \in \mathbb{Z}_n$ wie folgt:

- `KeyGen()` erzeugt zufällig zwei grosse Primzahlen p und q (k-Bit-Zahlen) und bestimmt den ***RSA-Modul*** $n = p \cdot q$. Dieser Modul und zusätzlich eine Zahl e mit der Eigenschaft

  $$1 < e < (p-1)(q-1) \quad \text{und} \quad \mathrm{ggT}\big(e, (p-1)\cdot(q-1)\big) = 1$$

  und ihre multiplikativ inverse Zahl $d = e^{-1}$ wird zurückgegeben.

  - Der RSA-Modul n und der Schlüssel e sind öffentlich

  - Die Primzahlen p, q und der Schlüssel d sind privat.
- $\mathtt{Enc'(e,P)} = P^e \bmod n$ (deterministische Variante)
- $\mathtt{Dec(d,C)} = C^d \bmod n$

Aufgabe 4: Zeigen Sie, dass RSA ein korrektes Kryptosystem ist.

Aufgabe 5: Gegeben sei der schwache öffentliche Schlüssel $e = 3$ mit dem RSA-Modul $n = 253$. Was ist der Klartext zum Chiffretext $C = 110$, wenn nicht probabilistisch verschlüsselt wurde?

Wie wird RSA randomisiert?

Als Standard wird ein ***OAEP-Verfahren***(Optimal Asymmetric Encryption) von Bellare und Rogaway verwendet.

- Mit einer passenden ***Expansionsfunktion*** $G : \mathbb{Z}_2^t \to \mathbb{Z}_2^l$ und einer ***Kompressionsfunktion*** $H : \mathbb{Z}_2^l \to \mathbb{Z}_2^t$, die beide öffentlich bekannt sind, wird ein Klartext $P \in \mathbb{Z}_2^l$ verschlüsselt. Mit einem ***uniform zufällig*** gewählten Bitstrom $r \in \mathbb{Z}_2^t$ wird der Bitstrom

$$B = \underbrace{P + G(r)}_{l\text{-Bits}} \bmod 2 \mid \underbrace{r + H(P + G(r))}_{t-\text{Bits}} \bmod 2$$

  berechnet und dann als Zahl mit dem Verschlüsselungsalgorithmus `Enc`$'()$ verschlüsselt.
- Mit dem Entschlüsselungsalgorithmus `Dec()` kann dann der Bitstrom B der Länge $l + t$ wieder hergestellt werden.

Aufgabe 6: Wie kann man aus B den Klartext P extrahieren?

Wie kann der RSA-Dec() Algorithmus beschleunigt werden?

Der Algorithmus `Dec()` rechnet in der modularen Arithmetik mit Modul $\boldsymbol{n}$. Es ist effizienter die Potenzen in der modularen Arithmetik mit Modul $\boldsymbol{p}$ und Modul $\boldsymbol{q}$ zu berechnen und dann mit dem ***Chinesischen Restsatz*** den Klartext zu rekonstruieren.

Weil die Module $\boldsymbol{p}$ und $\boldsymbol{q}$ beide Primzahlen sind, kann zudem die Berechnung mit dem ***kleinen Satz von Fermat*** vereinfacht werden.

Aufgabe 7: Führen Sie diese beschleunigte Berechnung für den Chiffretext $C = 119$ mit den Primzahlen $p = 11$ und $q = 23$ und dem privaten Schlüssel $d = 147$ durch.

Frage 3: *Was sind Zero-knowledge Proofs of Knowledge?*

Zero-Knowledge Proofs werden in Anwendungen eingesetzt, wo Vertrauen, Datenschutz und Sicherheit zugleich erforderlich sind. Zum Beispiel ist das ***Schnorr-Protokoll*** ein Zero-Knowledge-Proof-of-Knowledge Protokoll für den diskreten Logarithmus. Es wird in vielen modernen Systemen als Basis oder Bestandteil verwendet:

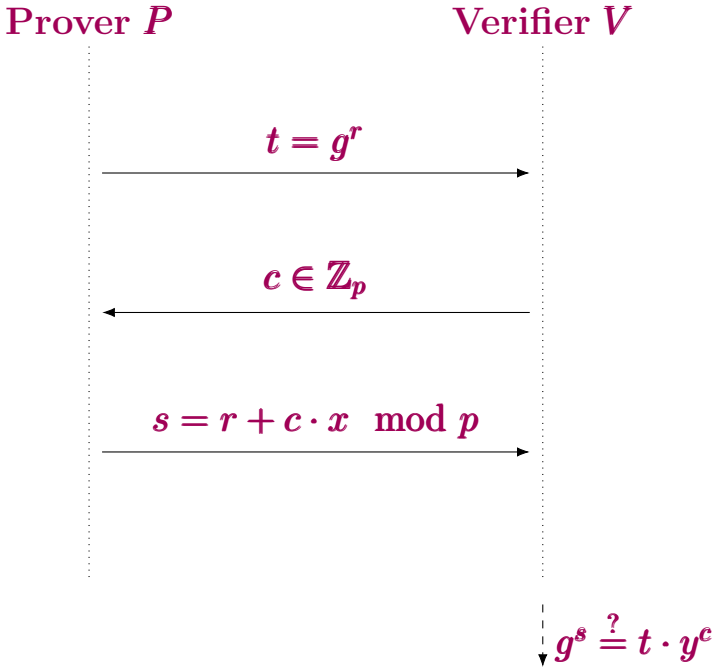
- Bitcoin Taproot verwendet eine Variante namens MuSig, die auf dem Protokoll der Schnorr-Signaturen basiert.
- Bei der anonymen Authentifizierung kann das Schnorr-Protokoll zeigen, dass jemand einen geheimen Schlüssel kennt (etwa in einem Login-System), ohne ihn preiszugeben.
- Das Schnorr Protokoll wird eingesetzt, um zu beweisen, dass ein Teilnehmer ehrlich rechnet, ohne seine Eingaben offenzulegen. Anwendung sind etwa in Wahlen, vertraulichen Abstimmungen, gemeinsamer Datenanalyse ohne Datenweitergabe.
- Systeme wie Idemix (IBM), AnonCreds etc. nutzen Zero-Knowledge Proofs für Datenschutz bei digitalen Ausweisen. Nutzer beweisen, dass sie bestimmte Attribute besitzen (etwa ein Alter über 18), ohne das Attribut vollständig offenzulegen.

Was macht das Schnorr-Protokoll?

Es handelt sich um einen ***Zero-Knowledge Proof of Knowledge***, bei dem ein ***Prover*** P einem Verifier V beweist, dass er einen diskreten Logarithmus $x \in \mathbb{Z}_p$ kennt, ohne ihn offenzulegen.

Definition 5: Sei G eine zyklische Gruppe mit $p \in \mathbb{P}$ Elementen und g ein Generator der Gruppe. Öffentlich bekannt ist ein Wert $y = g^x \in G$:

- ***Commitment***: P wählt uniform zufällig ein $r \in \mathbb{Z}_p$, berechnet $t = g^r$ und sendet t an V.
- ***Challenge***: V wählt uniform zufällig ein $c \in \mathbb{Z}_p$ und sendet c an P.
- ***Response***: P berechnet $s = r + c \cdot x \bmod p$ und sendet s an V.
- ***Verification***: V prüft, ob $g^s \stackrel{?}{=} t \cdot y^c$. Wenn ja, akzeptiert V.



Sicherheit: Niemand kann ohne Kenntnis von x ein gültiges s erzeugen, da r nicht rekonstruiert werden kann.

Zero-Knowledge: Der Verifier lernt nichts über x. Er sieht nur Zufallszahlen.

Wissenstransferfrei: Der Prover überzeugt den Verifier, ohne Wissen preiszugeben.

Aufgabe 8: Begründen Sie, warum das Schnorr-Protokoll korrekt ist.